Lab Standard: Scalable ML Experimentation with Hydra

Medical Computer Vision Lab (MCV Lab)

October 16, 2026

1 Purpose

This document defines the lab standard for using **Hydra** to manage machine learning experiments. Hydra is required for projects expected to scale across:

- multiple datasets / modalities
- multiple architectures
- multi-stage pipelines (pretrain → finetune → evaluate)
- multiple sites/domains
- HPC workloads (SLURM)

2 Why Hydra (vs argparse)

Argument parsers are a good baseline. However, Hydra provides better scalability:

H1. Hierarchical configs (clean separation of dataset/model/optimizer).

H2. Composable config groups (`model=resnet50`, `dataset=rsna`).

H3. Automatic run directories with saved configs.

H4. Built-in multirun sweeps (`-m`).

H5. Less experiment glue code and fewer hard-coded defaults.

3 Required Directory Structure

All Hydra-based training pipelines must follow this layout:

```
project/
  train.py
  configs/
    config.yaml
    dataset/
      rsna_trauma.yaml
      mimic_ecg.yaml
    model/
      resnet50.yaml
      vit_b16.yaml
    optim/
      adamw.yaml
      sgd.yaml
    trainer/
      default.yaml
```

4 Config Design Principles

4.1 Rule 1: No code edits for experiments

All experiment variations must be expressed as config overrides.

4.2 Rule 2: Separate concerns

- dataset configs must only contain dataset-related params
- model configs must only contain model-related params
- optim configs must only contain optimizer/scheduler params
- trainer configs must contain training loop settings (epochs, amp, ckpt)

4.3 Rule 3: Every run must be reproducible

Every run must:

- store the complete resolved config
- store launch command
- store outputs in a unique run directory

5 Minimal Hydra Training Entry Point

Save as `train.py`:

```
#!/usr/bin/env python3
from __future__ import annotations

import os
import random
from dataclasses import dataclass
from pathlib import Path
from typing import Optional

import numpy as np
import hydra
from omegaconf import DictConfig, OmegaConf

try:
    import torch
except ImportError:
    torch = None


def set_seed(seed: int, deterministic: bool = False) -> None:
    random.seed(seed)
    np.random.seed(seed)
    if torch is not None:
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        if deterministic:
            torch.backends.cudnn.deterministic = True
            torch.backends.cudnn.benchmark = False


@hydra.main(version_base=None, config_path="configs", config_name="
    config")
def main(cfg: DictConfig) -> None:
```

```
    # Hydra changes working directory into a run dir automatically
    run_dir = Path(os.getcwd())

    # Save resolved config
    (run_dir / "config_resolved.yaml").write_text(
        OmegaConf.to_yaml(cfg),
        encoding="utf-8",
    )

    # Reproducibility
    set_seed(int(cfg.seed), deterministic=bool(cfg.deterministic))

    print("=== RUN DIR ===")
    print(run_dir)

    print("\n=== CONFIG ===")
    print(OmegaConf.to_yaml(cfg))

    # TODO:
    # 1) build dataset based on cfg.dataset
    # 2) build model based on cfg.model
    # 3) build optimizer based on cfg.optim
    # 4) implement training loop using cfg.trainer
    print("\n[INFO] Training would start here.")


if __name__ == "__main__":
    main()
```

6 Hydra Base Config

Save as `configs/config.yaml`:

```
defaults:
  - dataset: rsna_trauma
  - model: resnet50
  - optim: adamw
  - trainer: default
  - _self_

seed: 0
deterministic: false

# optional global experiment tags
experiment:
  name: baseline
  project: mcv_lab
```

7 Example Config Groups

7.1 Dataset config

Save as `configs/dataset/rsna_trauma.yaml`:

```
name: rsna_trauma
data_dir: /datasets/rsna
num_workers: 4
```

```
target: any_injury
```

7.2 Model config

Save as `configs/model/resnet50.yaml`:

```
name: resnet50
pretrained: true
in_channels: 3
num_classes: 2
```

7.3 Optimizer config

Save as `configs/optim/adamw.yaml`:

```
name: adamw
lr: 1e-4
weight_decay: 1e-4
scheduler: cosine
```

7.4 Trainer config

Save as `configs/trainer/default.yaml`:

```
epochs: 30
batch_size: 16
amp: true
log_every: 50
save_every: 1
```

8 Standard Usage

8.1 Single run

```
python train.py dataset=rsna_trauma model=resnet50 optim=adamw trainer=
    default
```

8.2 Override parameters

```
python train.py model=resnet50 optim.lr=5e-5 seed=2
```

8.3 Multi-run sweep (Hydra multirun)

```
python train.py -m optim.lr=1e-3,1e-4,1e-5 seed=0,1,2
```

9 Lab Requirements for Hydra Projects

Before merging new Hydra projects:

- Must use `configs/` with config groups
- Must support `-m` sweeps
- Must save resolved config in the run folder
- Must never hardcode dataset/model params in `train.py`

10 Recommended Extensions

- Add W&B logging: log the Hydra resolved config
- Add `hydra.job.name` to label experiments
- Add `hydra.sweep.dir` for clean sweep directory naming